第 4-2 讲 • 从需求到设计（上）

> **模块 4 • 学会改代码** ｜ 建议用时 **40 分钟**（阅读 20 + 动手 20）
> 前置要求：完成 4-1（掌握安全流程）

本节目标

读完并做完这一讲，你能：

1. 拿到一个需求后，**先问清楚三个问题**，而不是直接开写。
2. 判断一个需求**属于哪种类型**（改界面？改逻辑？改数据？），从而估准工作量。
3. 用**影响面分析**列出一份准确的"要改的文件清单"。
4. 理解这次功能的一个**真实设计难点**：排序条件是写在 SQL 里的，怎么让它可配置。
5. **本讲不写代码**——只做设计。这是"改动越大越要设计"的实践。

一、本次任务：给记录列表加排序

1.1 需求原文

> **"我想让记录列表能按金额排序，而不只是按日期。"**

这就是真实需求的典型样子：一句话，没有细节。**产品经理（或者你自己）觉得说清楚了，其实没有。**

1.2 先问三个问题

任何需求拿到手，先问这三件事：

问题	为什么问	本次的答案
① **要什么？** 具体表现是什么	避免做错方向	让用户能选择"按金额排"还是"按日期排"，并且能切换升序/降序
② **影响谁？** 哪些地方要用	决定改动范围	只影响记录页的列表
③ **边界在哪？** 什么不做	**防止范围蔓延**	只做"按日期/按金额"两种，不做自定义排序、不做多列排序

第 ③ 个问题最重要，也最容易漏。

为什么：如果不明确"不做什么"，需求会自己长大：

```
"加排序" → "顺便按分类排吧" → "能不能拖拽调整顺序" → "排序要能保存到下次打开"
```

三天后你发现自己在做一个小型表格组件——而原始需求只是"想按金额看看哪笔花得多"。

记住这条：**每次动手前，写下"这次不做什么"。**

二、判断需求类型（决定工作量）

2.1 三种类型

回顾 3-3 讲和 3-4 讲学过的判断方法：

类型	特征	工作量	例子
A. 纯显示改动	数据已有，只是换个样子展示	**很小**（1 个文件）	加一列、改颜色、改文案
B. 交互/逻辑改动	数据已有，但要有新的"操作方式"	**中等**（3–5 个文件）	加筛选、加排序、加批量操作
C. 数据改动	要**存新的东西**	**较大**（6–10 个文件，含迁移）	加字段、加新表

2.2 本次属于哪一类

需求："按金额排序"。

判断过程：

问题	答案	推论
需要存新数据吗?	**不需要。**金额本来就存着	❌ 不是 C 类
数据够算吗?	**够。** `amount` 字段现成的	只改"取数据的方式"
是纯显示吗?	**不是。**用户要能**操作**（切换排序方式）	❌ 不是 A 类
→ 结论	**B 类：交互/逻辑改动**	预计改 **3–5 个文件**

这个判断的价值：你现在知道**大概要动几个文件**了，**心理有底，不会中途发现"怎么越改越多"。**

三、影响面分析（本讲核心）

3.1 先看现状：排序现在是怎么实现的

第一步永远是"看清楚现在是什么样"：

```
cd "C:\Users\34123\Desktop\记账簿"
Select-String -Path src\main\db\repositories\records.repo.ts -Pattern "ORDER BY" -Context
```

你会看到（真实代码）：

```
    ${where.sql}
    ORDER BY r.date DESC, r.created_at DESC      ← 排序写死在这里
    LIMIT ? OFFSET ?`
```

这是本讲最重要的发现：

> **排序条件是"硬编码"在 SQL 语句里的**——永远按日期倒序，用户改不了。

所以这个需求的核心，就是把"写死的排序"变成"可以传进来的排序"。

3.2 顺着数据流找要改的地方

用 3-2 讲的五站链路，**从后往前推**：

站点	文件	要改什么
① **契约**	`src\shared\types.ts`	在 `RecordQuery` 里加排序参数
② **仓储层**	`src\main\db\repositories\records.repo.ts`	把写死的 `ORDER BY` 改成根据参数拼
③ **接口层**	`src\main\ipc\records.ts`	可能要校验排序参数是否合法
④ **通道**	`src\preload\index.ts`	**不用改**（`list` 已经能传任意 query）
⑤ **界面**	`RecordsPage.tsx`	加排序选择的 UI，并放进 query

关键判断：**第 ④ 站不用改。**

为什么：看一下 preload 里的代码（2-2 讲学过）：

```
list: (query: RecordQuery) => ipcRenderer.invoke(IPC.recordsList, query)
```

它传的是整个 `query` 对象——所以只要 `RecordQuery` 类型变了，**它自动支持新参数，一行都不用改。**

这就是"好的设计"带来的好处：**契约变了，通道自动跟上。**

3.3 三个设计决策（这才叫"设计"）

到这里，改动清单清楚了。但还有三件事必须想清楚，否则会写出有 bug 的代码。

决策一：升序还是降序，要不要让用户选?

选项 A：只做"按金额排序"，固定降序（大额在前） **选项 B**：按金额 + 能切换升降序

我的建议：选 A（固定降序）。

理由：

理由	说明
真实需求是"看看哪笔花得多"	那句话里的"排序"本来就是想看大额，不是想研究升序
符合最小改动原则	只加一个选项，不加两个
以后可以再加	升序降序是独立的增量功能，不影响这次的结构

注意：这就是开头第 ③ 问的意义——**"这次不做什么"帮我砍掉了一半工作量。**

决策二：排序参数怎么设计（类型层面）

这是本讲最有技术含量的部分。

天真的设计：

```
sortBy?: string     // ❌ 用户传什么都行，拼进 SQL 就危险了
```

为什么危险：如果直接把 `sortBy` 拼进 SQL（`ORDER BY ${sortBy}`），用户传一个 `date; DROP TABLE records` 就能删库——**这就是 3-2 讲说的 SQL 注入。**

正确的设计：**限定取值范围。**

```
/** 排序方式：只允许这几种 */
export type RecordSort = 'date-desc' | 'amount-desc'

export interface RecordQuery {
  ...
  sort?: RecordSort     // ← 只能传这两种之一
}
```

好在哪：

好处	说明
类型层面就限死了	传别的值，TypeScript 直接报错
SQL 里是"查表"而不是"拼接"	仓储层用一个映射表把 `'amount-desc'` 换成固定 SQL 片段，**用户数据永远进不了 SQL 语句**
可扩展	以后要加"按分类排序"，加一个枚举值 + 一行映射

这是安全设计的通用思路：

> **不要拼接用户输入，要让用户从"允许的选项"里挑。**

同样的思路你在项目里见过：`type: 'income' | 'expense' | 'transfer'`（3-2 讲的校验就是检查它是不是这三种之一）。

决策三：默认值是什么

必须定默认值，因为：

- 老用户升级后，界面不会自动传这个参数
- 不传的时候要有合理行为

正确的默认：`'date-desc'`**（按日期倒序）**——

因为这是现在的行为。保持默认行为不变，意味着**升级到新版本后，列表看起来一模一样。**用户不会觉得"我的软件变了"。

这条原则叫"向后兼容"：**新功能不能改变老行为。**

四、写下来：本次的改动清单

设计阶段的产出就是这张表：

#	文件	改什么	类型
1	`src\shared\types.ts`	加 `RecordSort` 类型；`RecordQuery` 加 `sort?` 字段	契约
2	`src\main\db\repositories\records.repo.ts`	`ORDER BY` 根据 `sort` 参数选择	数据层
3	`src\renderer\src\features\records\RecordsPage.tsx`	加排序选择 UI（一个 `Segmented`）；把值放进 query；加进 `useMemo` 依赖	界面
—	`src\preload\index.ts`	**不用改**	—
—	`src\main\ipc\records.ts`	**不用改**（`list` 不校验 query 细节）	—
—	数据库	**不用改**（没有新数据）	—

三个文件，零数据库改动。

对比一下：如果这是一个"给记录加商家字段"的需求（C 类），就要动 8 个地方 + 一个迁移。**所以判断需求类型，能让你提前知道要花多少时间。**

五、动手任务（本讲不写代码，只做设计）

步骤 1：确认现状（排序是写死的）

```
Select-String -Path src\main\db\repositories\records.repo.ts -Pattern "ORDER BY" -Context
```

预期看到：`ORDER BY r.date DESC, r.created_at DESC`

记住这一行——它是这次要改的核心。

步骤 2：确认"通道层不用改"

```
Select-String -Path src\preload\index.ts -Pattern "recordsList"
```

预期看到：`list: (query: RecordQuery) => ipcRenderer.invoke(IPC.recordsList, query)`

思考：为什么它不用改？

答案：**它传的是整个 `query` 对象。** `query` 类型变了，它自动支持新字段。**如果当初 preload 写的是 `list: (from, to, keyword) => ...`（把每个参数列出来），那这次就要改它了。**

这是"设计好坏"的直观差别：**传对象 vs 列参数**，决定了以后加字段是"零成本"还是"要改三处"。

步骤 3：找出界面侧要改的位置

```
Select-String -Path src\renderer\src\features\records\RecordsPage.tsx -Pattern "useMemo" -
```

你要找到的是：**构造 `query` 对象的那段代码。**它真实长这样（`RecordsPage.tsx`）：

```
const query = useMemo(() => {
  const base = allTime ? {} : monthRange(month)
  return { ...base, keyword: keyword.trim() || undefined, limit: 500 }
}, [allTime, month, keyword])
```

逐行读懂它：

代码	意思		
`useMemo(() => {...}, [依赖])`	只在依赖变化时重新计算这个对象（2-4 讲的状态概念）		
`allTime ? {} : monthRange(month)`	如果选"全部时间"就不限日期，否则限定在当前月份		
`keyword: keyword.trim() | | undefined`	关键词为空时给 `undefined`（表示"这个条件不限制"）		
`limit: 500`	一次最多取 500 条		
`[allTime, month, keyword]`	**这三个变了才重算**		

找到之后回答：

- 你要在哪里加上 `sort`？→ `return { ...base, keyword: ..., limit: 500, sort }`
- `useMemo` **的依赖数组里要不要加** `sort`**？ → 必须加**

最后一问很关键：如果不把 `sort` 加进依赖数组，**用户切换排序时，query 不会重新计算——界面看起来"点了没反应"。**

真实后果长这样：

```
// ❌ 错误：依赖数组里没有 sort
const query = useMemo(() => {
  return { ...base, keyword: keyword.trim() || undefined, limit: 500, sort }
}, [allTime, month, keyword])        // ← sort 变了，这里不重算

// 用户点击「按金额」→ setSort('amount-desc') → 状态变了
// 但 query 没重算 → 发出去的请求还是旧的 sort → 列表纹丝不动
```

这是 React 最常见的 bug 之一（学名"闭包捕获了旧值"）。**你在这里提前知道它，比将来 debug 半小时划算。**

步骤 4：写下来

新建一个文本文件（或写在纸上），写下：

1. **这次做什么**：加"按日期/按金额"两种排序，默认按日期（保持不变）
2. **这次不做什么**：不做升序降序切换、不做拖拽排序、不做"记住上次选择"
3. **要改的文件**（3 个）：类型、仓储层、记录页
4. **不用的文件**：preload、IPC、数据库
5. **三个设计决策**：只做降序、用枚举类型不用字符串、默认保持日期倒序

这份东西就是你的"设计文档"。 下一讲照着它写代码。

为什么必须写下来：因为**脑子里的设计会变。**写着写着你会发现"等等，升序要不要做？"——**在纸上发现比在代码里发现便宜得多。**

卡住了怎么办

现象	原因	怎么办
找不到 `ORDER BY`	可能不止一处	搜 `ORDER BY` 全部结果，看哪一处是列表查询用的（第 172 行那个）
不确定 `useMemo` 依赖	正常，这是 React 的难点	记住规则：**用到的变量都要列进去**，包括新加的 `sort`
想不清楚要不要做升序	说明"边界"还没定	回到 1.2 节第 ③ 问：**真实需求是什么?**
觉得 3 个文件太多	这已经很少了	对比 C 类需求（8 个文件 + 迁移），B 类算轻的

六、自测题

1. 拿到需求后必须先问哪三个问题？为什么第 ③ 个最重要?

2. "按金额排序"属于哪一类需求？预计改几个文件?

3. 为什么 preload 这一层不用改？这反映了什么样的设计?

4. 排序参数为什么不能用 `string`，而要用 `'date-desc' | 'amount-desc'` 这样的枚举?

5. 为什么默认值必须是"按日期倒序"?

答案与解析

1. ① **要什么**（具体表现）② **影响谁**（改动范围）③ **边界在哪**（不做什么）。 → 第 ③ 个最重要，因为**不写清"不做什么"，需求会自己长大**（"加排序" → "加拖拽" → "加排序记忆"），最后你发现自己在做一个小型表格组件。

2. B 类（交互/逻辑改动）：不需要存新数据（金额已存在），但要有新的操作方式。 → 预计改 **3 个文件**：类型契约、仓储层、记录页。**不用改数据库、不用改 preload。**

3. 因为它的代码是 `list: (query) => invoke(..., query)` —— **传的是整个对象**，不是逐个列参数。 → 所以 `RecordQuery` 类型一变，它自动支持。**这体现了"传对象"比"列参数"更耐改动**——以后加字段零成本。

4. 因为**枚举类型在类型层面就限死了取值范围**，而且仓储层可以**用映射表把枚举值换成固定 SQL 片段**（查表，不是拼接）。 → 如果用 `string`，用户能传任意内容，拼进 SQL 就是**注入漏洞**（`date; DROP TABLE records`）。**原则：让用户从允许的选项里挑，绝不拼接用户输入。**

5. 因为**这保持和现在完全一样的行为**——老用户升级后，列表看起来没变。 → 这叫**向后兼容**：**新功能不能改变老行为。**如果默认改成"按金额排序"，用户升级后会以为"我的记录怎么乱了"。

七、术语表

术语	一句话解释	
需求分析	把一句模糊的话拆成"做什么、影响谁、不做什么"	
范围蔓延	需求在执行过程中悄悄变大	
向后兼容	新版本不改变旧行为，用户升级后无感	
枚举类型	限定取值范围，如 `'a' \	'b'`
硬编码 / Hardcode	把值写死在代码里，改不了	
映射表 / Map	用"查表"代替"拼接"，避免注入	
默认值 / Default	参数不传时使用的值	
依赖数组	React `useMemo` 等的"这些变量变了就重算"清单	
闭包陷阱	函数记住了旧变量值，导致"改了没反应"	

八、本讲与项目的对应

- 你查看的文件：`records.repo.ts`（ORDER BY 那行）、`preload\index.ts`、`RecordsPage.tsx`
- 你产出的东西：**一份改动清单**（下一讲要用）
- **本讲完全没有改代码**，不影响任何东西
- 上一讲：4-1 改代码的安全流程 ｜ 下一讲：**4-3 从需求到设计（下）**（照着清单把代码写出来并验收）

讲义版本：v1 ｜ 对应项目版本：0.1.0